KMS context 채우기 + LLM-judge 레이어 학습 노트

날짜: 2026-06-13 **파일**: `backend/m2/docs/NEXT_SESSION.md` **복습 예정**: D+1, D+3, D+7

한 줄 요약

m2(코드 리뷰 봇)가 지금은 도구만 쓰는데, "도서관에서 참고자료도 찾아보고 + AI 선생님한테 2차 검토도 받는" 기능을 추가하는 작업.

Part 1 — KMS context 채우기

비유 (중학생 수준)

지금 m2는 PR diff(제출된 코드 변경본)를 보고 **무조건 혼자서** Pylint·Bandit 같은 도구로 검사함. 문제집 풀 때 아무 참고자료 없이 푸는 것과 같음.

KMS(지식 관리 시스템, m0 담당)는 팀의 **사내 도서관**임. "N+1 쿼리 방지 가이드", "SQL 인젝션 패턴" 같은 문서가 저장돼 있음.

Part 1은: **PR 검사 시작 전에 도서관에서 관련 자료를 먼저 꺼내서 옆에 펴놓는 것**. 단, 도서관 문이 잠겨있어도(m0 미구현) 그냥 빈손으로 시작하면 됨(graceful).

개념

```
현재 흐름:
  POST /orchestrate/review
  → pr_diff 받음
  → 정적분석 + 보안스캔
  → m4 판정 → 응답

추가 후 흐름:
  POST /orchestrate/review
  → pr_diff 받음
  → [NEW] context 비어있으면 KMS에 "이 코드 관련 자료 줘" 요청
  → context 채운 뒤 정적분석 + 보안스캔
  → m4 판정 → 응답
```

`context`란 이런 3가지 보조정보:

- `architecture_docs` — "우리 서비스 구조 문서"
- `coding_conventions` — "팀 코딩 규칙"
- `previous_pr_issues` — "이 개발자가 전에 자주 틀린 것들 (N+1 쿼리, SQL 인젝션 등)"

왜 필요한가

지금 m2는 PR diff만 보고 판단함. context가 없으면:

- 팀 컨벤션 위반인지 모름 (도구는 문법만 봄)
- 이 개발자가 이 실수를 전에도 반복했는지 모름
- 서비스 아키텍처 맥락 없이 판단함

실제 구현 파일 (4개)

```
① backend/_shared/shared/contracts.py          ← ✅ 이미 완료 (스킵 가능)
   KmsSearchRequest / KmsSearchResult / KmsSearchResponse 이미 존재

② backend/m2/app/clients/kms_client.py         ← 신규 생성
   m0 POST /search 호출하는 함수
   M0_URL 없거나 m0가 404 내면 → None 반환 (graceful, 분석은 계속됨)
   m4_client.py 구조를 그대로 미러링

③ backend/m2/app/orchestrator.py               ← 수정
   run_m2_orchestration() 시작부에 kms_client.search() 호출 ~12줄 추가
   응답 있으면 req.context 채움, 없으면 그냥 진행

④ backend/m2/app/contract.py                   ← 수정
   M2OrchestrateResponse에 kms_used: bool = False 필드 추가
   (프론트/m4가 "이번 분석에 KMS 썼어?" 알 수 있게)
```

⚠️ 블로커 상황 (해결됨)

m0에 `/search` 엔드포인트가 **아직 없음**. 하지만 kms_client가 실패 시 `None` 반환하도록 만들어서 m2 분석 자체는 항상 돌아감. m0 담당자가 나중에 `/search` 구현하면 **코드 변경 없이** 자동 연결됨.

코드 예시 (우리 프로젝트)

```python
# app/clients/kms_client.py — m4_client.py 패턴 그대로
import os
import httpx
from shared.contracts import KmsSearchRequest, KmsSearchResponse

async def search(req: KmsSearchRequest) -> KmsSearchResponse | None:
    base_url = os.getenv("M0_URL") or os.getenv("M0_BASE_URL")
    if not base_url:
        return None  # graceful: 설정 없으면 스킵
    try:
        async with httpx.AsyncClient(timeout=10.0) as client:
            resp = await client.post(f"{base_url}/search", json=req.model_dump())
            resp.raise_for_status()
            return KmsSearchResponse(**resp.json())
    except Exception:
        return None  # graceful: 실패해도 분석은 계속
```

```python
# orchestrator.py — run_m2_orchestration 시작부 추가
from app.clients import kms_client
from shared.contracts import KmsSearchRequest

async def run_m2_orchestration(req):
    # context 비어있으면 KMS에서 채우기
    if not req.context.architecture_docs and not req.context.coding_conventions:
        query = "\n".join(req.pr_diff.splitlines()[:20])  # diff 앞 20줄로 질의
        kms_resp = await kms_client.search(KmsSearchRequest(query=query, top_k=3))
        kms_used = False
        if kms_resp and kms_resp.results:
            req.context.architecture_docs = [r.content for r in kms_resp.results]
            kms_used = True
    # 이후 기존 분석 흐름 그대로 ...
```

Part 2 — LLM-judge 레이어

비유

Part 1에서 참고자료를 꺼내왔는데, 지금 m2 도구들(Pylint, Bandit 등)은 그 참고자료를 **읽을 줄 모름**. 그냥 코드 문법만 체크하는 기계임.

Part 2는: **AI 선생님(Claude)을 추가해서 "도구가 찾은 이슈 + 팀 컨벤션 + 과거 실수 기록"을 함께 보고 2차 판정**하게 하는 것.

예시:

- Pylint가 "이 함수 복잡도 높음(Major)"이라고 했는데
- AI 선생님이 "팀 컨벤션 보니까 이건 허용된 패턴이고, 이 개발자가 전에 N+1 쿼리를 3번 반복했으니 그게 더 Critical" 이렇게 재판단

개념

```
현재: pr_diff → [도구] → issues 리스트 → m4

추가: pr_diff → [도구] → issues 리스트
                              ↓
                        [LLM-judge] ← context (컨벤션 + 과거이슈 + 아키텍처)
                              ↓
                        재판단된 issues 리스트 → m4
```

모델 라우팅 (비용 절약 전략):

이슈 심각도	사용 모델	이유
Critical	`claude-sonnet-4-6`	비쌈, 중요하니까 정확하게
Major	`claude-haiku-4-5-20251001`	저렴, 충분함

이슈 심각도	사용 모델	이유
Minor	LLM 스킵	도구 결과 그대로, 비용 절약

왜 필요한가

도구(Pylint 등)는 **규칙 기반**이라 맥락을 모름. LLM이 있어야:

- "이 팀에서는 이게 괜찮은 패턴" 판단 가능
- "이 개발자 반복 실수라 severity 올려야 함" 판단 가능
- suggestion도 팀 맥락에 맞게 보강 가능

실제 구현 파일

```
dev2/nodes/llm_judge.py     ← 이미 있는 원본 청사진
→ app/services/llm_judge.py 로 미러링 (타입만 IssueItem/IssueSummary에 맞춤)
```

핵심 패턴:

```python
# app/services/llm_judge.py 핵심 구조 (dev2 미러)
import os
from langchain_anthropic import ChatAnthropic

MODEL_ROUTING = {
    "Critical": "claude-sonnet-4-6",
    "Major": "claude-haiku-4-5-20251001",
    "Minor": None,   # LLM 스킵
}
LLM_TIMEOUT_SEC = 30.0
USE_MOCK = os.getenv("USE_MOCK_LLM", "true").lower() == "true"

async def judge_issues(issues, context, developer_level):
    if USE_MOCK:
        return _get_mock_response(issues)   # 개발/테스트: 가짜 응답

    # 실제 LLM 호출 (병렬)
    results = await asyncio.gather(
        *[_judge_single_issue(issue, developer_level, context) for issue in
issues],
        return_exceptions=True
    )
    # 예외 발생한 이슈는 원본 그대로 유지 (graceful)
    return [
        original if isinstance(result, Exception) else result
        for original, result in zip(issues, results)
    ]
```

graceful 처리 흐름:

```
LLM 미설정(USE_MOCK=true)  → mock 응답 반환
LLM 타임아웃               → 원본 도구 결과 유지
JSON 파싱 실패             → 재시도 1회 → 실패 시 원본 유지
예외 발생                  → 해당 이슈만 원본 유지, 나머지는 계속
```

핵심 개념 3가지

1. **Graceful degradation** — 외부(m0 KMS, Claude API)가 죽어도 m2 분석 자체는 항상 돌아감. 외부 실패 = "없는 것처럼" 처리.
2. **Mock/실제 전환** — `USE_MOCK_LLM` 환경변수 하나로 코드 변경 없이 전환. 개발 중엔 무조건 `true`.
3. **additive 원칙** — 기존 코드 최소 수정, 새 파일 추가 위주. m0 코드 건드리지 않음.

작업 순서 요약

```
Part 1 (작고 안전, 먼저):
  ✅ 1. contracts.py KmsSearch* 추가    → 이미 완료, 스킵
  □  2. kms_client.py 신규 생성
  □  3. orchestrator.py 연결 (~12줄)
  □  4. contract.py kms_used 필드 추가
  □  5. 테스트 2경로 (정상 채움 / None 시 빈 context)

Part 2 (큰 작업, 나중):
  □  1. pyproject.toml langchain-anthropic 추가
  □  2. analyzer.py context 파라미터 추가
  □  3. app/services/llm_judge.py 신규 (dev2 미러)
  □  4. 프롬프트 빌더 (context 3필드 주입)
  □  5. orchestrator 연결
  □  6. 테스트 (USE_MOCK_LLM=true 기준)
  □  7. MODULE_CONTRACT.md uses_kms 갱신
```

복습 퀴즈

1. Q: m0 `/search`가 없는데 Part 1을 왜 지금 구현해도 되나? A: (kms_client가 실패 시 None 반환 → graceful, 분석은 계속됨)

2. Q: Minor 이슈에 LLM-judge를 쓰지 않는 이유는? A: (비용 절약 — Minor는 도구 결과 그대로 써도 충분)

3. Q: `kms_used: bool` 필드를 응답에 추가하는 이유는? A: (m4나 프론트가 "이번 분석에 KMS 참고했는지" 알 수 있게)

4. Q: LLM-judge에서 예외 발생 시 어떻게 처리되나? A: (해당 이슈만 원본 도구 결과 유지, 나머지 분석은 계속)

5. Q: context 3가지 필드는 무엇인가? A: (architecture_docs / coding_conventions / previous_pr_issues)